

Bug Resuelto: El Bot Mostraba

{{full_name}} / {{custom_question_X}} Literales

Fecha: 30 de junio de 2026

Severidad:  **ALTA** (imagen de marca + leads de Meta)

Estado:  **RESUELTO** (capa de código) ·  requiere acción en Meta para personalización real

Tipo: Handoff Meta → Landing → Zobot (SalesIQ)

1. Síntoma Reportado

En una prueba real desde el anuncio de Meta, el bot saludó así (ver captura del cliente):

¡Hola {{full_name}}! 🙌 Soy el asistente de Kaven Sports. Vi que ya nos compartiste tu información para {{custom_question_1}} — ¡mil gracias! ... Como mencionaste alrededor de {{custom_question_2}} personas, y que tu diseño esta “{{custom_question_4}}”, con fecha estimada para {{custom_question_3}}...

Es decir, el bot mostró los **tokens de plantilla literales** en lugar de los datos reales del lead. Además, el nombre del visitante en el chat aparecía como {{full_name}}.

2. Causa Raíz

El flujo de handoff funciona así:

```
Anuncio Meta → abre URL de la landing con parámetros
    ?source=meta&name={{full_name}}&sport={{custom_question_1}}...
    ↓
meta-lead-context.js captura los parámetros
    ↓
POST /meta-context (backend) → guarda el contexto
    ↓
Handler v18 (Deluge) lo recupera y arma el saludo
```

El problema: Meta **NO sustituye** tokens como {{full_name}} o {{custom_question_1}} cuando abre la URL de un **sitio web externo**. Esos tokens solo funcionan **dentro del ecosistema de Meta** (respuestas automáticas en Messenger, pantalla de agradecimiento del propio Instant Form), **NO** en un redirect a una página web.

Resultado: los tokens llegaban **literales** a la landing, se guardaban tal cual, y el bot los reproducía en el saludo.

⚠ **Limitación de plataforma:** Los **Formularios Instantáneos (Instant Forms / Lead Ads)** de Meta **no pueden** pasar las respuestas del formulario a la URL de un sitio web. No existe una función nativa para inyectar `{{campo}}` en el botón de “Ver sitio web”. Los únicos macros que Meta sí sustituye en URLs son de campaña (p.ej. `{{campaign.name}}`, `{{ad.id}}`, `{{site_source_name}}`), **nunca** datos del formulario.

3. Solución Implementada (3 capas de blindaje)

Se blindó el sistema para que **NUNCA** vuelva a mostrar `{{...}}` roto. Si los valores llegan como tokens sin sustituir, se descartan y el bot cae a su **flujo normal** (saluda limpio y captura los datos en el sitio).

Capa 1 — Landing: `landing/meta-lead-context.js`

La función `clean()` ahora detecta y elimina cualquier token `{{...}}` antes de guardar/inyectar:

```
if (v.indexOf('{{') >= 0 && v.indexOf('}}') >= 0) {
  v = v.replace(/\{\{[\^]*\}\}/g, " ").replace(/\s+/g, " ").trim();
}
```

Capa 2 — Backend: `server.js` endpoint `POST /meta-context`

Nueva función `stripMetaTemplateTokens()` aplicada a todos los campos de texto. Si tras limpiar no queda **ni nombre ni teléfono** reales, el contexto **NO se guarda** (el bot usa su flujo normal):

```
function stripMetaTemplateTokens(v) { ... }
// name: stripMetaTemplateTokens(b.name), etc.
if (!data.name && !data.phone) return res.json({ ok:false, error:"missing_name_and_phone" });
```

Capa 3 — Handler Deluge: `message_handler_v18_meta_aware.deluge`

Tras parsear el payload, se descarta cualquier campo con `{{`. Si no quedan nombre ni teléfono reales, se **degrada** `isMetaLead = false` → se muestra el formulario normal y NO se envía un saludo vacío/roto. Además el saludo ahora es seguro sin nombre (¡Hola! en lugar de ¡Hola !).

4. Pruebas Realizadas

- ✓ Sintaxis: `node -c server.js / meta-lead-context.js` → OK
- ✓ `stripMetaTemplateTokens`: 5/5 casos (tokens → "", texto real intacto)
- ✓ `POST /meta-context` con tokens → RECHAZADO (`missing_name_and_phone`)
- ✓ `POST /meta-context` con datos reales → ACEPTADO + recuperable por GET
- ✓ `name real + sport token` → guarda name, limpia sport

Resultado: El bot ya **no puede** mostrar `{{...}}` bajo ninguna ruta.

5. Importante: Personalización Real vs. Captura de Leads

Aspecto	Estado
¿El bot vuelve a mostrar <code>{{...}}</code> ?	✅ NO (blindado)
¿El bot saluda profesional aunque no haya datos?	✅ Sí (flujo normal)
¿Los leads de Meta llegan al CRM?	✅ Sí (vía LeadChain nativo, ya corregido el 30-jun)
¿El bot puede saludar por nombre desde un Instant Form?	❌ NO es posible vía URL (limitación de Meta)

Opciones para el dueño del negocio

1. **Recomendada (más simple):** Dejar el blindaje. El bot saluda limpio y ayuda con su flujo normal; los datos del lead se siguen guardando en CRM por la integración nativa. Quitar los `{{...}}` de la URL del anuncio y dejar solo `?source=meta` (así el bot sabe que viene de Meta).
2. **Si se quiere que el bot “conozca” al lead:** usar anuncios **Click-to-Messenger** (la conversación se queda en Messenger, donde Meta Sí pasa el contexto), en lugar de redirigir a la web.

6. Pendiente de Publicar (manual)

1. **Backend:** push a GitHub → Render redeploy automático (incluye `/meta-context` blindado).
2. **Landing:** desplegar la versión actualizada de `meta-lead-context.js`.
3. **SalesIQ:** publicar `message_handler_v18_meta_aware.deluge` actualizado.
4. **Meta (recomendado):** quitar los tokens `{{...}}` de la URL del anuncio; dejar solo `?source=meta`.

7. Archivos Modificados

- `landing/meta-lead-context.js` — `clean()` descarta tokens `{{...}}`
- `server.js` — `stripMetaTemplateTokens()` + sanitización en `POST /meta-context`
- `salesiq/message_handler_v18_meta_aware.deluge` — descarta tokens, degrada a flujo normal, saludo seguro sin nombre

Estado Final: ✅ El bot ya no muestra placeholders rotos. La captura de leads a CRM sigue funcionando por la integración nativa. La personalización por nombre desde un Instant Form no es posible vía URL (limitación de Meta) — ver opciones en la sección 5.